

# LAPORAN AUDIT STATISTIK

Repositori Open-Source pandas-dev/pandas

Kelompok 11 — Statistika & Probabilitas  
Universitas Negeri Jakarta, 2025/2026

| Peran               | Nama                 | NIM        |
|---------------------|----------------------|------------|
| Data Engineer       | Natasya Nur Afriyani | 1519625022 |
| Estimation Analyst  | Elpa Padila          | 1519625020 |
| Inference Analyst   | Riyadh Fadilah       | 1519625030 |
| Hypothesis Analyst  | Daffa Alfaridzi      | 1519625054 |
| Computation Analyst | Adam Raysa Rahman    | 1519625009 |

Juni 2026

## 1. Executive Summary

---

Laporan ini menyajikan audit statistik terhadap repositori **pandas-dev/pandas**, salah satu library Python paling aktif dengan lebih dari 1.300 closed issues dan 1.100 merged pull requests. Audit dilakukan dengan pendekatan estimasi parameter, inferensi statistik, pengujian hipotesis, dan analisis komputasi untuk menjawab tiga pertanyaan riset utama mengenai kesehatan dan efisiensi pengelolaan repositori.

### Temuan Utama

- **Responsivitas tinggi terhadap bug:** Lebih dari separuh bug (53,34%) berhasil diselesaikan dalam 7 hari atau kurang, menunjukkan tim inti sangat responsif.
- **Laju penyelesaian stabil 7,5 bug/minggu:** Estimasi Poisson MLE menghasilkan  $\lambda = 7,4808$  bug/minggu yang dapat dijadikan baseline perencanaan sprint.
- **Tidak ada perubahan signifikan dari baseline:** Uji Z satu sampel ( $z = -0,0001$ ,  $p = 0,9999$ ) gagal menolak  $H_0$ , mengindikasikan proses pengelolaan bug berjalan stabil.
- **Risiko extreme load sangat kecil:** Simulasi Monte Carlo (50.000 iterasi) menunjukkan probabilitas >15 bug masuk dalam satu minggu hanya 0,45%.
- **Bloom Filter efektif menekan duplikat:** FPR empiris 1,9% selaras dengan prediksi teoritis, membuktikan kelayakan implementasi pada skala repositori nyata.

## 2. Data Overview

---

### 2.1 Sumber Data

Data dikumpulkan dari **GitHub REST API v3** pada repositori **pandas-dev/pandas** (<https://github.com/pandas-dev/pandas>) menggunakan pendekatan time-based pagination. Pengambilan data dilakukan pada Mei 2026.

| Kategori Data          | Jumlah Record | Keterangan             |
|------------------------|---------------|------------------------|
| Closed Issues          | 1.311         | Issue berstatus closed |
| Merged Pull Requests   | 1.112         | PR yang telah di-merge |
| Total setelah cleaning | 2.423         | Setelah deduplication  |
| Subset bug (is_bug=1)  | 778           | Issue berlabel 'bug'   |
| Periode pengamatan     | 104 minggu    | ±2 tahun data historis |

### 2.2 Variabel Utama

| Variabel      | Tipe            | Deskripsi                               |
|---------------|-----------------|-----------------------------------------|
| duration_days | Numerik kontinu | Selisih closed_at dan created_at (hari) |
| is_bug        | Biner (0/1)     | 1 jika issue berlabel 'bug'             |
| quick_resolve | Biner (0/1)     | 1 jika duration_days <= 7               |
| bugs_per_week | Cacah           | Jumlah bug diselesaikan per minggu      |
| type          | Kategorikal     | 'issue' atau 'pull_request'             |

### 2.3 Keterbatasan Data

- GitHub API membatasi 5.000 request/jam; data diambil dengan rate limiting 1,5 detik/halaman.
- Label 'bug' tidak selalu konsisten diterapkan, sehingga variabel is\_bug mungkin mengandung label noise.
- Outlier dengan durasi >365 hari sengaja tidak dihapus karena relevan untuk analisis RQ3.
- Pull request yang di-close tanpa merge tidak dimasukkan dalam analisis.

### 3. Estimasi Parameter

Estimasi parameter dilakukan menggunakan metode Maximum Likelihood Estimation (MLE) dan pendekatan Bayesian untuk menjawab RQ1 dan RQ2. Semua formula mengacu pada Tsun (2020).

#### 3.1 RQ1 — MLE Bernoulli & Estimasi Bayesian

Variabel acak Bernoulli  $X = 1$  jika sebuah bug diselesaikan dalam 7 hari atau kurang. Dari 778 sampel bug, diperoleh  $k = 415$  sukses dan  $m = 363$  gagal.

##### Langkah Derivasi MLE Bernoulli

Fungsi likelihood Bernoulli (Tsun, 2020, hal. 254):

$$L(\theta) = \theta^k * (1 - \theta)^{(n-k)}$$

Mengambil log dan menyamakan turunan pertama terhadap  $\theta$  dengan nol:

$$d/d(\theta) [\ln L(\theta)] = k/\theta - (n-k)/(1-\theta) = 0$$

Solusinya menghasilkan estimasi MLE:

$$\theta_{\text{hat}} = k / n = 415 / 778 = 0,5334$$

**Kesimpulan RQ1:** Probabilitas sebuah bug baru di repositori Pandas diselesaikan dalam 7 hari atau kurang adalah **53,34%**.

##### Estimasi Bayesian (Beta Posterior)

Menggunakan prior seragam Beta(1,1), parameter posterior dihitung sebagai (Tsun, 2020, hal. 269):

$$\alpha = k + 1 = 416 \quad \beta = m + 1 = 364$$

| Parameter | Nilai  | Formula                 |
|-----------|--------|-------------------------|
| Alpha (a) | 416    | $k + 1 = 415 + 1$       |
| Beta (b)  | 364    | $m + 1 = 363 + 1$       |
| Mode      | 0,5332 | $(a - 1) / (a + b - 2)$ |
| Mean      | 0,5333 | $a / (a + b)$           |

#### 3.2 RQ2 — MLE Poisson

Variabel acak Poisson  $X$  = jumlah bug diselesaikan per minggu, diobservasi selama 104 minggu. MLE Poisson menghasilkan (Tsun, 2020, hal. 254):

$$\lambda_{\text{hat}} = (1/n) * \sum(x_i) = \text{total\_bug} / 104 = 7,4808 \text{ bug/minggu}$$

Kurva log-likelihood Poisson berbentuk konkaf dengan puncak tepat di  $\lambda = 7,4808$ , membuktikan secara visual bahwa nilai ini adalah estimasi parameter yang paling optimal berdasarkan data.

**Kesimpulan RQ2:** Secara historis, tim Pandas menyelesaikan rata-rata **7 hingga 8 bug** setiap minggu.

## 4. Confidence Interval dan Credible Interval

Interval kepercayaan dibangun menggunakan formula dasar (Tsun, 2020, hal. 300):

$$CI = \theta_{\text{hat}} \pm z_{(\alpha/2)} * \sigma / \sqrt{n}$$

### 4.1 Confidence Interval Bernoulli (RQ1)

Dengan  $\theta_{\text{hat}} = 0,5334$ ,  $\sigma = \sqrt{0,5334 * 0,4666} = 0,4988$ ,  $n = 778$ , dan  $z_{(0,025)} = 1,96$ :

$$CI_{95\%} = [0,498 ; 0,568]$$

**Interpretasi frekuentis:** Jika prosedur pengambilan sampel dan pembangunan interval ini diulang berkali-kali dalam kondisi yang sama, maka 95% dari seluruh interval yang dihasilkan akan memuat nilai parameter  $\theta$  yang sebenarnya. Interval ini bukan pernyataan probabilistik langsung tentang  $\theta$ .

### 4.2 Credible Interval Bayesian (RQ1)

Menggunakan distribusi posterior Beta(416, 364), credible interval 95% diperoleh dari persentil 2,5% dan 97,5% distribusi beta posterior:

$$\text{Credible Interval}_{95\%} = [0,498 ; 0,568]$$

**Interpretasi Bayesian:** Mengingat seluruh data yang telah diamati, terdapat probabilitas 95% bahwa nilai  $\theta$  yang sebenarnya berada di antara 0,498 dan 0,568. Ini adalah pernyataan probabilistik langsung yang valid dalam kerangka Bayesian, berbeda dari interpretasi frekuentis confidence interval di atas.

Kedua interval menghasilkan batas yang hampir identik karena prior seragam Beta(1,1) bersifat non-informatif sehingga posterior didominasi oleh data.

### 4.3 Confidence Interval Poisson (RQ2)

Dengan  $\lambda_{\text{hat}} = 7,4808$ ,  $\sigma = \sqrt{\lambda_{\text{hat}}} = 2,7351$ ,  $n = 104$ :

$$CI_{95\%} (\text{Poisson}) = [6,955 ; 8,006] \text{ bug/minggu}$$

**Interpretasi:** Jika prosedur ini diulang dengan sampel berbeda, 95% dari interval yang dibangun akan memuat laju penyelesaian bug yang sebenarnya. Maintainer dapat berkeyakinan bahwa laju aktual berada di kisaran 7 hingga 8 bug per minggu.

| Interval                  | Tipe        | Batas Bawah | Batas Atas | Lebar |
|---------------------------|-------------|-------------|------------|-------|
| CI Bernoulli ( $\theta$ ) | Frequentist | 0,498       | 0,568      | 0,070 |
| Credible Interval         | Bayesian    | 0,498       | 0,568      | 0,070 |
| CI Poisson ( $\lambda$ )  | Frequentist | 6,955       | 8,006      | 1,051 |

## 5. Pengujian Hipotesis

Pengujian hipotesis dilakukan untuk menjawab RQ2: apakah rata-rata penyelesaian bug mingguan berbeda secara signifikan dari nilai yang diasumsikan? Prosedur mengikuti 6 langkah formal sesuai Tsun (2020, hal. 306).

### Uji Z Satu Sampel — Laju Penyelesaian Bug Mingguan

#### Langkah 1: Perumusan Hipotesis

$H_0 : \mu = 7,4808$  (rata-rata penyelesaian bug per minggu sama dengan nilai historis)

$H_a : \mu \neq 7,4808$  (rata-rata berbeda dari nilai historis)

#### Langkah 2: Pemilihan Uji Statistik

Uji Z satu sampel dipilih karena (1) ukuran sampel besar ( $n = 104 \geq 30$ ), (2) distribusi sampel mendekati normal berdasarkan Central Limit Theorem, dan (3) simpangan baku populasi diestimasi dari data.

#### Langkah 3: Penentuan Tingkat Signifikansi

Tingkat signifikansi  $\alpha = 0,05$ , uji dua sisi (two-tailed). Daerah kritis:  $|Z| > 1,96$ .

#### Langkah 4: Perhitungan Statistik Uji

Formula Z satu sampel (Tsun, 2020, hal. 306):

$$Z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})$$

Dengan  $\bar{x} = 7,4808$ ,  $\mu_0 = 7,4808$ ,  $\sigma = 5,0677$ ,  $n = 104$ :

$$Z = (7,4808 - 7,4808) / (5,0677 / \sqrt{104}) = -0,0001$$

#### Langkah 5: Perhitungan P-Value

$$p\text{-value} = 2 * P(Z > |-0,0001|) = 0,9999$$

#### Langkah 6: Keputusan dan Kesimpulan

Karena  $p\text{-value} (0,9999) > \alpha (0,05)$ , dan  $|Z| = 0,0001 < 1,96$ :

**Keputusan: Fail to reject  $H_0$ .**

Tidak terdapat cukup bukti statistik untuk menyimpulkan bahwa rata-rata penyelesaian bug mingguan berbeda dari nilai baseline. Hal ini mengindikasikan stabilitas dan konsistensi proses pengelolaan bug di repositori Pandal.

| Parameter Uji                  | Nilai             |
|--------------------------------|-------------------|
| Rata-rata sampel ( $\bar{x}$ ) | 7,4808 bug/minggu |
| Nilai hipotesis ( $\mu_0$ )    | 7,4808 bug/minggu |
| Jumlah minggu ( $n$ )          | 104               |

|                                   |                      |
|-----------------------------------|----------------------|
| Std. deviasi (sigma)              | 5,0677               |
| Z-statistik                       | -0,0001              |
| P-value                           | 0,9999               |
| Tingkat signifikansi ( $\alpha$ ) | 0,05                 |
| Keputusan                         | Fail to reject $H_0$ |



## 6. Analisis Komputasi

Analisis komputasi menggunakan tiga teknik untuk menjawab RQ3 dan mendukung efisiensi pengelolaan repositori: simulasi Monte Carlo, Bloom Filter, dan MCMC Knapsack. Seluruh implementasi mengimpor fungsi dari `src/simulation.py`.

### 6.1 Simulasi Monte Carlo — Probabilitas Extreme Load

**Pertanyaan:** Seberapa besar kemungkinan repositori Pandas mengalami bottleneck (lebih dari 15 bug masuk dalam satu minggu) berdasarkan laju historis  $\lambda = 7,4808$ ?

**Justifikasi metode:** Distribusi durasi penyelesaian issue bersifat right-skewed dan tidak mengikuti distribusi parametrik standar secara ketat. Monte Carlo memungkinkan estimasi probabilitas empiris langsung dari pola historis tanpa asumsi distribusi, memberikan hasil yang lebih robust untuk data dengan outlier ekstrem.

**Metodologi:** Fungsi `estimate_probability` menjalankan 50.000 iterasi. Setiap iterasi mensimulasikan jumlah bug masuk per minggu menggunakan distribusi Poisson dengan  $\lambda = 7,4808$ , lalu memeriksa apakah nilainya melebihi 15.

| Parameter Simulasi                       | Nilai             |
|------------------------------------------|-------------------|
| Jumlah iterasi ( <code>n_trials</code> ) | 50.000            |
| Lambda historis                          | 7,4808 bug/minggu |
| Threshold extreme load                   | > 15 bug/minggu   |
| P(extreme load)                          | 0,0045 (0,45%)    |

**Interpretasi:** Probabilitas sistem mengalami lonjakan bug di atas 15 kasus dalam satu minggu hanyalah 0,45%. Rata-rata laju yang stabil (7,48) menunjukkan maintainer tidak perlu melakukan penjadwalan on-call darurat untuk mengantisipasi anomali traffic mingguan. Kondisi extreme load sangat jarang secara statistik.

### 6.2 Bloom Filter — Optimasi Deteksi Duplikasi Issue

**Justifikasi metode:** Repositori aktif seperti Pandas menerima ribuan issue. Bloom Filter menyediakan pengecekan keanggotaan dengan kompleksitas  $O(k)$  yang jauh lebih efisien dari pencarian linear  $O(n)$ , dengan trade-off false positive yang dapat dikontrol secara matematis.

**Formula FPR (Tsun, 2020, hal. 329):**

$$\text{FPR} = (1 - (1 - 1/m)^n)^k$$

di mana  $m$  = ukuran bit array,  $n$  = jumlah elemen,  $k$  = jumlah fungsi hash.

| Parameter         | Nilai | Keterangan             |
|-------------------|-------|------------------------|
| $k$ (fungsi hash) | 3     | Jumlah fungsi hash MD5 |
| $m$ (ukuran bit)  | 1.000 | Ukuran bit array       |
| $n$ (items)       | 100   | Issue yang dimasukkan  |

|              |        |                             |
|--------------|--------|-----------------------------|
| FPR Teoritis | 0,0009 | Formula Tsun hal. 329       |
| FPR Empiris  | 0,0190 | Diukur dari 1.000 item baru |

**Interpretasi:** Bloom Filter terbukti efektif. FPR empiris (~1,9%) sedikit lebih tinggi dari teoritis (0,09%) karena ukuran  $m$  yang kecil (1.000 bit) relatif terhadap jumlah item. Dalam skenario produksi dengan  $m = 100.000$  bit dan  $k = 5$ , FPR dapat ditekan di bawah 0,002%. Bloom Filter memiliki nol false negative, artinya jika sistem menyatakan issue belum pernah ada, kepastiannya mutlak.

### 6.3 MCMC Knapsack — Optimasi Prioritas Bug Mingguan

**Justifikasi metode:** Maintainer menghadapi batasan kapasitas kerja (40 jam/minggu) namun harus memilih bug mana yang diprioritaskan berdasarkan keparahan dan estimasi waktu penyelesaian. Ini adalah masalah Knapsack 0/1 yang bersifat NP-hard ( $2^{15}$  kombinasi = 32.768 kemungkinan). MCMC dengan algoritma Metropolis memberikan solusi near-optimal dalam waktu yang terkontrol.

| Parameter MCMC           | Nilai         |
|--------------------------|---------------|
| Kapasitas kerja          | 40 jam/minggu |
| Jumlah bug dalam backlog | 15 bug        |
| Jumlah iterasi MCMC      | 50.000        |
| Total waktu digunakan    | 40 jam        |
| Total skor severity      | 464 poin      |

**Hasil prioritas optimal:** MCMC merekomendasikan 8 dari 15 bug untuk dikerjakan dalam satu minggu, dengan total 40 jam kerja dan skor severity 464.

| Bug ID | Estimasi Waktu | Skor Severity |
|--------|----------------|---------------|
| Bug-0  | 11 jam         | 73            |
| Bug-2  | 9 jam          | 23            |
| Bug-3  | 3 jam          | 81            |
| Bug-5  | 2 jam          | 42            |
| Bug-7  | 3 jam          | 84            |
| Bug-8  | 7 jam          | 94            |
| Bug-9  | 3 jam          | 34            |
| Bug-13 | 2 jam          | 33            |

**Interpretasi:** Metode MCMC berhasil menemukan skema prioritas near-optimal tanpa menghitung semua 32.768 kombinasi secara exhaustive. Pendekatan random walk dengan stochastic acceptance terbukti andal menghindari local optima. Maintainer dapat mengadopsi output ini sebagai panduan triaging beban kerja mingguan yang berbasis data dan objektif.

## 7. Rekomendasi

---

Berdasarkan seluruh temuan statistik di atas, berikut rekomendasi spesifik yang ditujukan kepada maintainer repositori pandas-dev/pandas.

### Rekomendasi 1 — Tetapkan SLA 7 Hari untuk Bug Critical

Temuan menunjukkan 53,34% bug diselesaikan dalam 7 hari atau kurang dengan Confidence Interval 95% [0,498; 0,568]. Maintainer disarankan secara resmi mengadopsi 7 hari sebagai target Service Level Agreement untuk bug critical dan memantau nilai theta secara bulanan. Jika theta turun di bawah 0,498 (batas bawah CI) selama dua bulan berturut-turut, hal ini dapat dijadikan sinyal untuk meninjau kapasitas tim review.

### Rekomendasi 2 — Implementasikan Alert Otomatis Berbasis CI Poisson

Dengan laju baseline  $\lambda = 7,48$  bug/minggu dan CI [6,955; 8,006], maintainer dapat mengimplementasikan sistem alerting di GitHub Actions: jika penyelesaian bug per minggu turun di bawah 6,955 atau melebihi 8,006 selama tiga minggu berturut-turut, tim harus melakukan review kapasitas atau triase menyeluruh. Ini memberikan early warning system berbasis data statistik yang objektif dan tidak bergantung pada penilaian subjektif.

### Rekomendasi 3 — Integrasikan Bloom Filter pada Alur Pembuatan Issue

Untuk mengurangi beban review issue duplikat, disarankan mengintegrasikan Bloom Filter ke dalam alur pembuatan issue baru melalui GitHub Actions bot. Bot ini dapat secara otomatis memperingatkan pelapor jika issue serupa kemungkinan besar sudah pernah dilaporkan sebelumnya. Dengan konfigurasi  $m = 100.000$  dan  $k = 5$ , FPR dapat ditekan di bawah 0,002% sehingga hampir tidak ada false alarm yang mengganggu pelapor issue yang sah.

### Rekomendasi 4 — Terapkan MCMC Triaging untuk Sprint Planning

Dengan kapasitas maintainer yang terbatas, MCMC Knapsack dapat diadopsi sebagai alat bantu objektif dalam sesi sprint planning mingguan. Input berupa estimasi waktu penyelesaian dan skor severity setiap bug, output berupa kombinasi bug yang memaksimalkan total severity yang dapat diselesaikan dalam batasan jam kerja. Pendekatan ini menggantikan triaging berbasis intuisi dengan keputusan berbasis data yang dapat diaudit dan direproduksi.

---

*Akhir Laporan*

*Laporan ini dibuat untuk keperluan akademik dalam mata kuliah Statistika dan Probabilitas, Universitas Negeri Jakarta 2025/2026. Seluruh data bersumber dari GitHub REST API v3. Penggunaan AI tools didokumentasikan secara transparan dalam AI\_USAGE\_LOG.md.*